Traffic Sign Classification Using CNN

Project Overview

This project implements a Deep Learning model for Traffic Sign Classification using Convolutional Neural Networks (CNNs) with the German Traffic Sign Recognition Benchmark (GTSRB) dataset.

The main goal of the project is to classify traffic signs into 43 different categories using image processing and deep learning techniques.

The project also compares the performance of two different optimizers:

- Adam Optimizer
- SGD (Stochastic Gradient Descent) Optimizer

The model is trained using PyTorch and evaluated using training accuracy, validation accuracy, and loss curves.

Objective of the Project

The objective of this project is to:

- Build a CNN model capable of recognizing traffic signs.
- Improve image classification accuracy using data preprocessing and augmentation.
- Compare different optimization techniques.
- Evaluate model performance visually using graphs.
- Test the trained model on unseen images.

Dataset Used

Dataset Name: German Traffic Sign Recognition Benchmark (GTSRB)

Dataset Link: https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign

The dataset contains:

- 43 traffic sign classes
- Thousands of training images
- Separate training and testing folders

Examples of traffic signs:

- Speed limit signs
- Stop signs

- No entry signs
- Warning signs
- Traffic direction signs

Project Workflow

The project workflow is divided into several main stages:

1. Dataset Loading
2. Data Preprocessing
3. Data Augmentation
4. Building CNN Model
5. Training the Model
6. Validation and Evaluation
7. Comparing Optimizers
8. Visualization of Results
9. Testing on New Images

Step 1: Dataset Loading

The dataset is loaded from Google Drive after extracting the ZIP file.

Main operations performed:

- Mounting Google Drive
- Extracting dataset files
- Verifying dataset paths
- Reading training and testing folders

Dataset folders:

- Train Folder
- Test Folder

Step 2: Data Preprocessing

Before feeding images into the neural network, preprocessing operations are applied.

Preprocessing Operations

Resize Images

All images are resized to:

32 × 32 pixels

This ensures that all images have the same dimensions.

Convert Images to Tensor

Images are converted into tensors so they can be processed by PyTorch.

Normalize Images

Normalization improves training stability and convergence.

Step 3: Data Augmentation

Data augmentation is used to improve model generalization and reduce overfitting.

Techniques used:

- Random Rotation
- Random Horizontal Flip

Benefits:

- Increases dataset diversity
- Helps the model learn robust features
- Improves performance on unseen data

Step 4: Dataset Splitting

The training dataset is divided into:

Dataset	Percentage
Training Set	80%
Validation Set	20%

Purpose of validation dataset:

- Monitor model performance during training
- Detect overfitting
- Evaluate generalization ability

Step 5: CNN Model Architecture

The project uses a custom Convolutional Neural Network.

CNN Layers Used

Convolution Layers

Used for extracting image features.

The model contains multiple convolution blocks.

Batch Normalization

Improves training speed and stability.

ReLU Activation Function

Introduces non-linearity into the network.

Max Pooling

Reduces spatial dimensions and computational cost.

Fully Connected Layers

Used for final classification.

Dropout Layer

Reduces overfitting by randomly disabling neurons during training.

CNN Architecture Summary

Layer Type	Purpose
Conv2D	Feature extraction
BatchNorm2D	Training stability
ReLU	Activation
MaxPool2D	Dimensionality reduction
Dropout	Reduce overfitting
Linear Layer	Classification

Step 6: Training Process

The model is trained using:

- Cross Entropy Loss Function
- Backpropagation
- Gradient Descent Optimization

Training settings:

Hyperparameter	Value
Batch Size	64
Learning Rate	0.001
Epochs	10
Number of Classes	43

The project automatically detects whether CUDA (GPU) is available.

Step 7: Optimizers Comparison

Two experiments were performed.

Experiment 1: CNN + Adam Optimizer

Adam optimizer combines:

- Momentum
- Adaptive Learning Rate

Advantages:

- Faster convergence
- Better performance in many deep learning tasks
- More stable training

Experiment 2: CNN + SGD Optimizer

SGD updates weights gradually using gradient descent.

Advantages:

- Simpler optimization method
- Lower memory usage

- Can generalize well in some cases

Step 8: Model Evaluation

The project evaluates the model using:

Training Accuracy

Measures performance on training data.

Validation Accuracy

Measures model generalization.

Training Loss

Shows how well the model is learning.

Validation Loss

Helps detect overfitting.

Visualization

The project visualizes:

- Adam Accuracy Curve
- Adam Loss Curve
- SGD Accuracy Curve
- SGD Loss Curve

These graphs help analyze:

- Model learning behavior
- Convergence speed
- Overfitting or underfitting

Step 9: Testing on New Images

After training, the model is tested on unseen traffic sign images.

Testing process:

1. Read image
2. Apply preprocessing
3. Pass image through trained model
4. Predict traffic sign class
5. Display predicted class

This demonstrates the real-world usability of the model.

Expected Results

The expected outputs of the project include:

- Successful traffic sign classification
- High training and validation accuracy
- Visualization graphs for accuracy and loss
- Comparison between Adam and SGD optimizers
- Predicted class labels for test images

Advantages of the Project

- Applies real-world computer vision techniques
- Demonstrates CNN implementation from scratch
- Uses image augmentation techniques
- Compares optimization algorithms
- Provides visualization for better analysis
- Can be extended for autonomous driving applications

Challenges Faced

Some possible challenges during the project:

- Overfitting
- Large training time
- Dataset imbalance
- GPU memory limitations
- Hyperparameter tuning

Real-World Applications

Traffic sign classification is used in:

- Autonomous vehicles
- Driver assistance systems
- Smart transportation systems
- Road safety monitoring
- Intelligent traffic management

Conclusion

This project successfully demonstrates how Convolutional Neural Networks can be used for traffic sign recognition.

The project covers the complete deep learning pipeline:

- Data preprocessing
- Data augmentation
- CNN architecture design
- Model training
- Evaluation
- Visualization
- Testing

The comparison between Adam and SGD optimizers also helps understand the impact of optimization algorithms on model performance.

Overall, the project provides a strong practical implementation of deep learning and computer vision concepts using PyTorch.

References

- PyTorch Documentation
- Torchvision Documentation
- GTSRB Dataset
- Deep Learning Research Papers